

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Факультет безопасности информационных технологий

Дисциплина:

«Криптографические методы обеспечения информационной безопасности»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №6

«Модель протокола защищенного соединения»

Выполнили:

Бардышев Артём Антонович,
студент группы N3346



(подпись)

Проверил:

Чешев Никита Игоревич,
инженер, ФБИТ

(отметка о выполнении)

(подпись)

Санкт-Петербург

2026 г.

СОДЕРЖАНИЕ

Введение	3
1 Подготовка программного обеспечения.....	4
2 Анализ раунда хэш-функции / имитовставки в Cryptool 2.....	5
3 Алгоритм генерации кода аутентификации сообщений (HMAC) в Cryptool 2	6
4 Эксперименты с хэшированием в OpenSSL ('openssl dgst').....	7
4.1 Хэширование строки (сообщения).....	7
4.2 Хэширование файла	7
4.3 Анализ свойств хэш-функций	7
5 Генерация HMAC в OpenSSL ('openssl dgst -hmac').....	8
6 Работа с асимметричными ключами RSA и симметричным ключом AES.....	9
6.1 Генерация пары ключей RSA	9
6.2 Генерация случайного симметричного ключа AES-128.....	9
6.3 Шифрование и расшифрование симметричного ключа RSA.....	9
6.4 Генерация и проверка подписи RSA для зашифрованного сообщения	9
7 Шифрование и дешифрование сообщения симметричным шифром ('openssl enc') ..	11
8 Генерация и проверка HMAC для зашифрованного сообщения	12
9 Альтернативный вариант: собственная реализация хэш-функции или имитовставки	13
Заключение.....	14
Список использованных источников.....	15

ВВЕДЕНИЕ

Целью лабораторной работы является изучение подходов к применению криптографических примитивов в рамках протоколов защищённых соединений.

В ходе работы исследуются взаимодействие хэш-функций, кодов аутентификации сообщений (HMAC), симметричных и асимметричных шифров, а также механизмов электронной подписи на примере утилиты OpenSSL.

Практическая часть имитирует ключевые шаги реальных протоколов (например, TLS): генерацию и передачу симметричного ключа посредством асимметричного шифрования, использование HMAC для контроля целостности и подлинности сообщений, а также применение хэш-функций для вычисления дайджестов.

1 ПОДГОТОВКА ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

1. Установить пакет OpenSSL:

- в Linux (например, Kali): через менеджер пакетов (``sudo apt install openssl``);
- в Windows: установить дистрибутив с поддержкой OpenSSL (либо воспользоваться WSL/Kali).

2. Проверить корректность установки командой:

`openssl version`

2 АНАЛИЗ РАУНДА ХЭШ-ФУНКЦИИ / ИМИТОВСТАВКИ В CRYPTOOOL 2

(по заданию: Templates → Hash Functions → MD5 / SHA-1 / SHA-2 / SHA-3 или шаблон имитовставки)

1. Запустить Cryptool 2 и открыть один из шаблонов:

- `Templates → Hash Functions → MD5` (или SHA-1 / SHA-2 / SHA-3);
- либо шаблон имитовставки (HMAC).

2. Визуализировать первый раунд работы алгоритма:

- отметить начальные значения внутренних переменных (например, A, B, C, D для MD5 или H0...H7 для SHA-2);
- проследить, как сообщение разбивается на блоки и как формируется первый блок.

3. Для выбранной функции подробно разберем:

- процесс инициализации начальных констант (A, B, C, D / H0...H7);
- используемые раундовые константы (таблица K);
- операции в одном раунде: побитовые сдвиги, циклические сдвиги, сложение по модулю 2^{32} , нелинейные побитовые функции.

3 АЛГОРИТМ ГЕНЕРАЦИИ КОДА АУТЕНТИФИКАЦИИ СООБЩЕНИЙ (НМАС) В CRYPTOOOL 2

(по заданию: Templates → Hash Functions → НМАС)

1. В Cryptool 2 открыть шаблон `Templates → Hash Functions → НМАС`.

2. Проанализировать, из каких блоков состоит схема:

- блоки для подготовки ключа (обрезка/дополнение до размера блока B);
- внутренний хэш (inner hash) и внешний хэш (outer hash);
- операции XOR с константами `ipad` и `opad`.

3. На основе шаблона записать общую формулу НМАС:

$$\text{НМАС}(K, M) = H((K' \oplus \text{opad}) \parallel H((K' \oplus \text{ipad}) \parallel M)),$$

где `K` — ключ, приведённый к длине блока хэш-функции, `ipad` и `opad` —

стандартные заполнители, `H` — выбранная хэш-функция (например, SHA-256).

```
C:\Users\ops-h>openssl version
OpenSSL 3.2.0 23 Nov 2023 (Library: OpenSSL 3.2.0 23 Nov 2023)

C:\Users\ops-h>echo hello > msg.txt

C:\Users\ops-h>openssl dgst -md5 msg.txt
MD5(msg.txt)= 6dd4566eb245627b49f3abb7e4502dd6

C:\Users\ops-h>openssl dgst -sha256 msg.txt
SHA2-256(msg.txt)= f873eef4f852e335da367d76ce7f1973c15b8ffebf532b064df4bc691cd51a87

C:\Users\ops-h>openssl dgst -sha512 msg.txt
SHA2-512(msg.txt)= b76ad68c3b3e3867d95d7162196287747ab30ef147759a096177efc8e7e187ae7087bdfdac5b5cf5d67217e21bb1a00ae852a
bbca9a8982dddc068d7e58bd8
```

Рисунок 1 – создание хешей для сообщения

4 ЭКСПЕРИМЕНТЫ С ХЭШИРОВАНИЕМ В OPENSSL (‘OPENSSL DGST’)

4.1 Хэширование строки (сообщения)

1. Выбрать произвольное текстовое сообщение, например:

```
echo -n "Hello, cryptography!" > msg.txt
```

2. Вычислить хэши разными алгоритмами:

```
openssl dgst -md5 msg.txt  
openssl dgst -sha256 msg.txt  
openssl dgst -sha512 msg.txt
```

3. Зафиксировать результаты (длину и вид хэша).

4.2 Хэширование файла

1. В качестве файла можно использовать `msg.txt` или любой другой файл (например, небольшую тестовую запись).
2. Аналогично вычислить хэш-значения теми же алгоритмами.

4.3 Анализ свойств хэш-функций

1. Изменить входное сообщение на 1 символ или 1 бит (например, добавить точку или изменить одну букву) и пересчитать хэши.
2. Сравнить старые и новые хэши:
 - показать, что даже при минимальном изменении входа выход существенно меняется (лавинный эффект);
 - отметить, что длина хэша не зависит от длины входного сообщения.

5 ГЕНЕРАЦИЯ HMAC В OPENSSL ('OPENSSL DGST -HMAC')

1. Выбрать ключ (например, строку `"secretkey"` или более сложный ключ) и использовать его с флагом `-hmac`:

```
openssl dgst -md5 -hmac "secretkey" msg.txt
openssl dgst -sha256 -hmac "secretkey" msg.txt
openssl dgst -sha512 -hmac "secretkey" msg.txt
```

2. Повторить вычисления с другими ключами разной длины (короткий ключ, длинная случайная строка) и убедиться, что OpenSSL принимает ключи разной длины, самостоятельно приводя их к нужному размеру.

В отчёте:

- перечислить использованные команды и варианты ключей;
- отметить, что формат и длина HMAC зависят от выбранной хэш-функции, а не от длины ключа;
- сделать вывод о гибкости HMAC по отношению к длине ключа.

6 РАБОТА С АСИММЕТРИЧНЫМИ КЛЮЧАМИ RSA И СИММЕТРИЧНЫМ КЛЮЧОМ AES

Задача: сгенерировать пару RSA-ключей, с их помощью зашифровать случайный симметричный ключ AES, затем расшифровать его и использовать для подписи/проверки.

6.1 Генерация пары ключей RSA

```
openssl genrsa -out rsa_priv.pem 2048
openssl rsa -in rsa_priv.pem -pubout -out rsa_pub.pem
```

В отчёте:

- указать длину ключа (например, 2048 бит);
- кратко описать формат файлов `rsa\_priv.pem` и `rsa\_pub.pem`.

6.2 Генерация случайного симметричного ключа AES-128

Например, можно сгенерировать 16 байт случайных данных в hex-формате:

```
openssl rand -hex 16 > aes.key
```

6.3 Шифрование и расшифрование симметричного ключа RSA

1. Шифрование симметричного ключа на открытом ключе:

```
openssl rsautl -encrypt -pubin -inkey rsa_pub.pem -in aes.key -out
aes.key.enc
```

2. Расшифрование симметричного ключа закрытым ключом:

```
openssl rsautl -decrypt -inkey rsa_priv.pem -in aes.key.enc -out aes.key.dec
```

3. Сравнить файлы `aes.key` и `aes.key.dec` (например, через `diff` или побайтовый вывод) и убедиться, что ключ восстановлен без искажений.

6.4 Генерация и проверка подписи RSA для зашифрованного сообщения

1. Вычислить хэш от файла `aes.key.enc`:

```
openssl dgst -sha256 -out aes.key.enc.sha256 aes.key.enc
```

2. Сгенерировать подпись при помощи закрытого ключа:

```
openssl rsautl -sign -inkey rsa_priv.pem -in aes.key.enc.sha256 -out
aes.key.enc.sha256.sig
```

3. Проверить подпись, используя открытый ключ:

```
openssl rsautl -verify -pubin -inkey rsa_pub.pem -in aes.key.enc.sha256.sig -
out aes.key.enc.sha256.ver
```

4. Убедиться, что файл ``aes.key.enc.sha256.ver`` совпадает с исходным ``aes.key.enc.sha256``.

7 ШИФРОВАНИЕ И ДЕШИФРОВАНИЕ СООБЩЕНИЯ СИММЕТРИЧНЫМ ШИФРОМ ('OPENSSL ENC')

1. Подготовить небольшой текстовый файл, например:

```
echo "Example of symmetric encryption with AES-128-CBC" > plaintext.txt
```

2. Зашифровать файл с помощью AES-128 в режиме CBC, используя ключ и/или пароль:

```
openssl enc -aes-128-cbc -salt -in plaintext.txt -out ciphertext.bin -pass  
file:./aes.key
```

(в качестве альтернативы можно использовать пароль через '-pass pass:...').

3. Расшифровать файл:

```
openssl enc -d -aes-128-cbc -in ciphertext.bin -out decrypted.txt -pass  
file:./aes.key
```

4. Сравнить 'plaintext.txt' и 'decrypted.txt' и убедиться, что данные совпадают.

8 ГЕНЕРАЦИЯ И ПРОВЕРКА НМАС ДЛЯ ЗАШИФРОВАННОГО СООБЩЕНИЯ

1. Сгенерировать НМАС для файла `ciphertext.bin` с тем же ключом, что использовался при шифровании (или с отдельным ключом):

```
openssl dgst -sha256 -hmac "same_secret_key" -out ciphertext.bin.hmac  
ciphertext.bin
```

или с ключом из файла:

```
openssl dgst -sha256 -mac HMAC -macopt hexkey:$(cat aes.key) -out  
ciphertext.bin.hmac ciphertext.bin
```

2. Для проверки целостности можно повторно вычислить НМАС и сравнить результаты:

- либо с помощью флага `-verify` (если используется поддерживаемый формат),
- либо просто с помощью утилиты `diff` / сравнения файлов.

9 АЛЬТЕРНАТИВНЫЙ ВАРИАНТ: СОБСТВЕННАЯ РЕАЛИЗАЦИЯ ХЭШ-ФУНКЦИИ ИЛИ ИМИТОВСТАВКИ

Если выбирается альтернативный вариант из ТЗ, необходимо:

1. Выбрать конкретный алгоритм (например, SHA-1, SHA-256, SHA-3, ГОСТ 34.11 или HMAC-SHA256).
2. Реализовать две процедуры:
 - генерация хэш-суммы (или имитовставки);
 - проверка хэш-значения (повторный расчёт и сравнение).
3. Протестировать реализацию на нескольких сообщениях, показать:
 - совпадение с эталонными значениями (например, с OpenSSL);
 - чувствительность к малым изменениям входных данных.

ЗАКЛЮЧЕНИЕ

В ходе лабораторной работы были исследованы основные криптографические примитивы, применяемые в протоколах защищённого соединения: хэш-функции, коды аутентификации сообщений (HMAC), симметричные и асимметричные шифры, а также электронная подпись.

На практике были отработаны навыки вычисления хэш-значений и HMAC с помощью OpenSSL, генерации и использования пары ключей RSA для защиты симметричного ключа AES и подписи зашифрованных данных, а также шифрования и дешифрования сообщений симметричными алгоритмами. Полученные результаты демонстрируют, как различные криптографические примитивы комбинируются между собой для обеспечения конфиденциальности, целостности и аутентичности данных в современных сетевых протоколах.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Официальная документация OpenSSL: ``https://www.openssl.org/docs/``.
2. OpenSSL Cookbook by Ivan Ristić (доступные онлайн-материалы по использованию OpenSSL).
3. FIPS 180-4: Secure Hash Standard (SHS).
4. RFC 2104: HMAC: Keyed-Hashing for Message Authentication.
5. Учебно-методические материалы кафедры по дисциплине «Криптографические методы обеспечения информационной безопасности» (ТЗ к лабораторной работе 6).